# IN 2211
# Object-Oriented Analysis and Design

**Lesson 02: Introduction to UML**


1

## Content

- Introduction to UML
- Why UML for Modeling?
- Diagram Taxonomy
- Introduction to Different UML Diagram Types

2

## Introduction

- Well-defined and expressive notation is important to the process of software development.
- A standard notation;
  - Makes it possible for an analyst or developer to describe a scenario or formulate an architecture.
  - Clearly communicate those decisions to others.
- E.g., Electronics - Circuit Diagrams

3

## What is a Model?

- A model provides blueprint of the system.
- It is an abstract representation of a system.
- This model representation promotes better understanding.
- Building a model is essential for communication of a project team.

Advantages:

- Helps in visualizing the system to be developed
- Used as template to construct a proposed system
- Helps to master the complex system
- Used to document the decision made
- Easy to capture and precise the requirements by all stakeholders

4

## Unified Modeling Language (UML)

- The primary modeling language used to;
  - Analyze
  - Specify
  - Design software systems
- An industry-standard graphical language for;
  - Specifying
  - Visualizing
  - Constructing
  - Documenting the artifacts of software systems

5

## Unified Modeling Language (UML)

- **Unified**
- **A Single model**
  - Can apply to almost all phases in software development life cycle
  - UML combined the best from object-oriented software modeling methodologies that were in existence during the early 1990's

**Modelling**

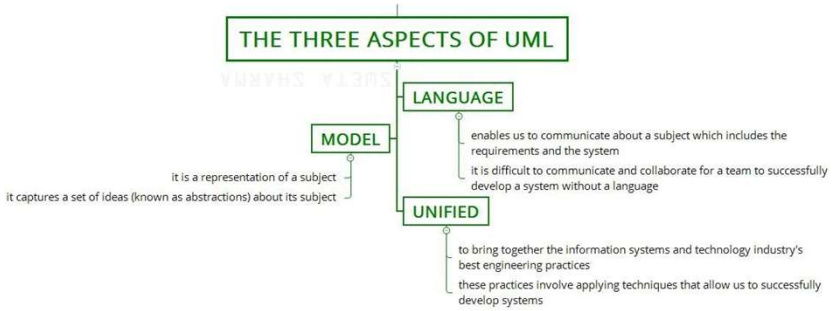
All creative disciplines use some form of modeling as part of the creative process.

**Language**

- A graphical language that follows a **precise syntax**.
- UML 2.0 is the most recent version
- UML is standardized.
- Content is controlled by the Object Management Group (OMG), a consortium of companies.

6

## Unified Modeling Language (Cont..)

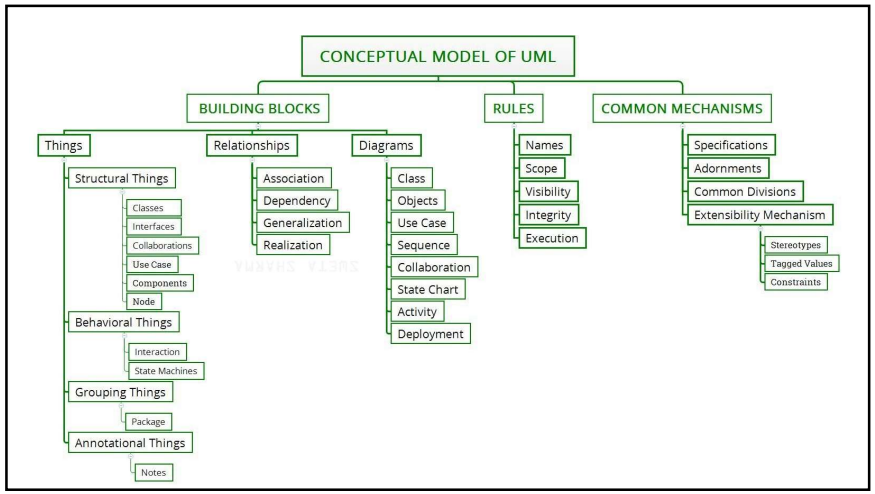



7

## A Conceptual Model of UML

- A conceptual model can be defined as a model which is made of concepts and their relationships.
- A conceptual model is the first step before drawing a UML diagram.
- It helps to understand the entities in the real world and how they interact with each other.
- As UML describes the real- time systems, it is very important to make a conceptual model and then proceed gradually.
- The conceptual model of UML can be mastered by learning the following three major elements;
  - UML building blocks
  - Rules to connect the building blocks
  - Common mechanisms of UML

8




9

## Bits from History

- Object-Oriented modeling languages made their appearance in the late 70's.
- Smalltalk was the first widely used Object-Oriented language.
- As the usefulness of Object-Oriented programming became undeniable, more Object-Oriented modeling languages began to appear.
- By the start of the 90's there was a flood of modeling languages, each with its own strengths and weaknesses.
- In 1994 the UML effort officially began as a collaborative effort between Booch and Rumbaugh.
- The goal of UML is to be a comprehensive modeling language (all things to all people) that will facilitate communication between all members of the development effort.

10

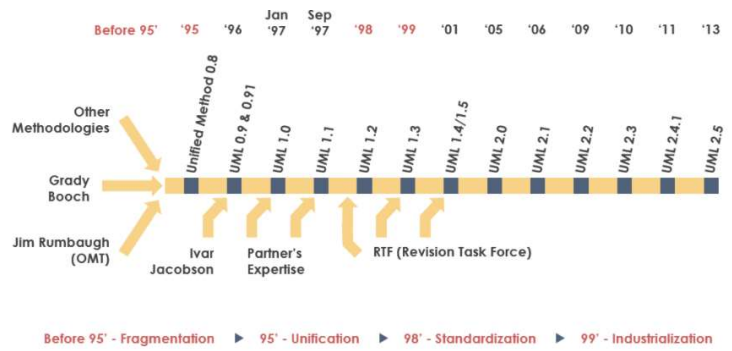



Figure: Evolution of UML

11

## Why UML for Modeling?

- Use graphical notation
  - communicate more clearly than natural language (imprecise) and code(too detailed)
- Helps acquire an overall view of a system
- Not depend on any other language or technology
- A simplified view of reality
  - facilitates the design and implementation of object-oriented software systems
- A language for documenting design
- Provides a record of what has been built
- Useful for bringing new programmers up to speed
- Describes what a system is supposed to do
  - But doesn't tell how to implement the system

12

## Advantages of UML

- The software system would **be professionally designed and documented** before it is coded.
  - You will know exactly what you are getting, in advance.
- Since system design comes first, **reusable code is easily spotted** and coded with the highest efficiency.
  - You will have lower development costs.
- **Logic 'holes' can be spotted in the design drawings**. Your software will behave as you expect it to.
  - There are fewer surprises.
- The overall **system design will dictate the way the software is developed**.
  - The right decisions are made before you are married to poorly written code.
  - Again, your overall costs will be less.

13

## Advantages of UML (Cont..)

- UML lets us see the big picture. We can develop more memory and processor efficient systems.
- When we come back to make modifications to your system, it is much easier to work on a system that has UML documentation.
- If you should find the need to work with another developer, the UML diagrams will allow them to get up to speed quickly in your custom system.
- If we need to communicate with outside contractors or even your own programmers, it is much more efficient.

14

## Diagram Taxonomy

- System complexity is driven by;
  - The number and organization of elements in the system - ***structure***
  - The manner in which all these elements collaborate to perform their function - ***behavior***
- UML diagrams can be classified into two groups:
  - Behavioral diagrams
  - Structural diagrams

15

## Diagram Taxonomy (Cont..)

- **Behavioral Diagrams**
  - Events happen dynamically in all software-intensive systems:
    - Objects are created and destroyed
    - Objects send messages to one another in an orderly fashion
    - External events trigger operations on certain Objects
  - Different types of UML diagrams describe the behavior of the system
    - Use case diagram
    - Activity diagram
    - State machine diagram
    - Interaction diagrams (A set of Diagrams)
      - Sequence diagram
      - Communication diagram
      - Interaction overview diagram
      - Timing diagram

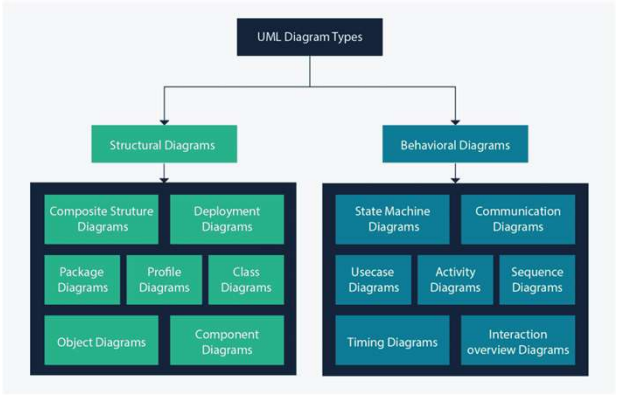
16

## Diagram Taxonomy (Cont..)

- **Structural Diagrams**
  - Used to show the static structure of elements in the system
  - It depicts
    - Architectural organization of the system
    - Physical elements of the system
    - domain-specific elements
  - The UML structural diagrams include the following:
    - Class diagram
    - Component diagram
    - Deployment diagram
    - Object diagram
    - Composite structure diagram
    - Package diagram

17

## Diagram Taxonomy (Cont..)




18

## BEHAVIORAL DIAGRAMS

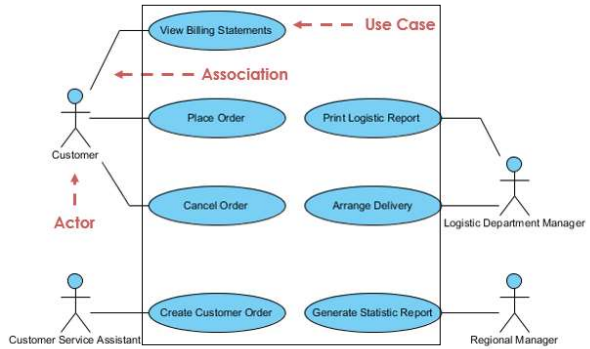
19

## Use Case Diagram

- What is a Use Case Diagram ?
  - A description of a system's behavior from a user's standpoint
    - Used for gathering system requirements from a user's point of view
  - In graphical representations of use cases a symbol for the actor is used.
    - **Actor**
      - Actor is the entity that initiates the use case. It can be a person or another system
    - **Use Case**
      - The action initiated by the Actor

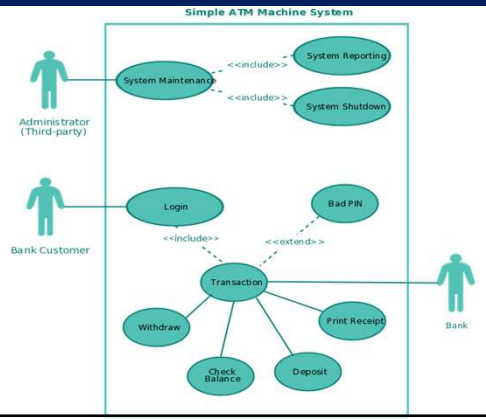
20

## Use Case Diagram (Cont..)




21

## Use Case Diagram (Cont..)



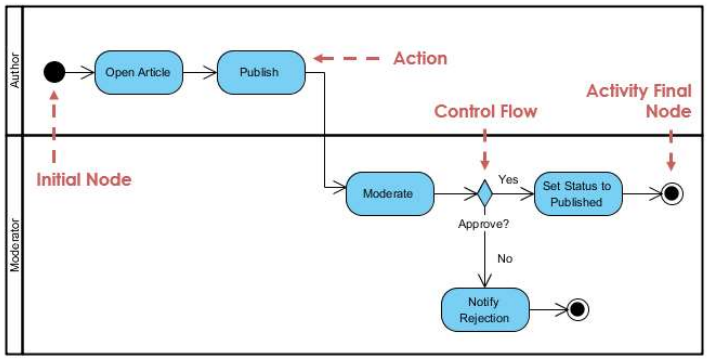

22

## Activity Diagram

- What is an Activity Diagram?
  - A visual depictions of the flow of activities, whether in a system, business, workflow, or other process
- Focus on the activities that are performed and who (or what) is responsible for the performance of those activities
- Elements of an activity diagram
  - Action nodes
  - Control nodes
  - Object nodes

23

## Activity Diagram (Cont..)



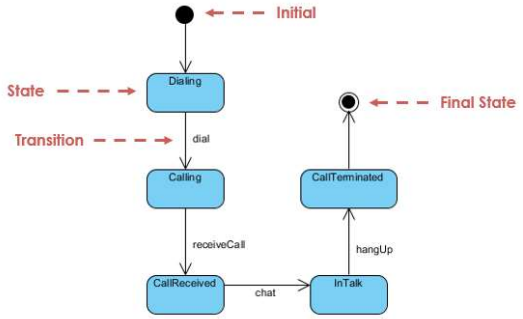

24

## State Machine Diagram

- At any given time, an object is in a particular state.
- State diagrams represent;
  - these states
  - their changes during time
- Every state diagram;
  - Starts with a symbol that represents start state
  - Ends with symbol for the end state

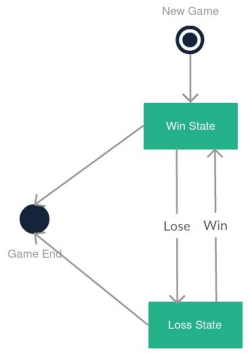
25

## State Machine Diagram (Cont..)




26

## State Machine Diagram (Cont..)




27

## Interaction Diagrams

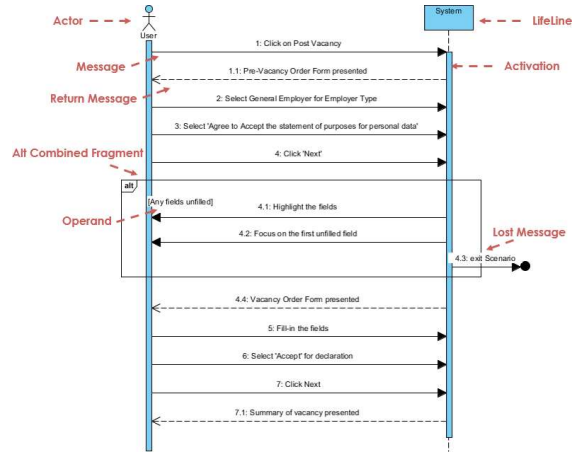
- The purposes of interaction diagrams are to visualize the interactive behavior of the system
- Visualizing interaction is a difficult task
  - In UML, there are four different types of Interaction Diagrams to capture the interactions from different perspectives
    - Sequence diagram
    - Communication diagram
    - Interaction overview diagram
    - Timing diagram

28

## Sequence Diagram

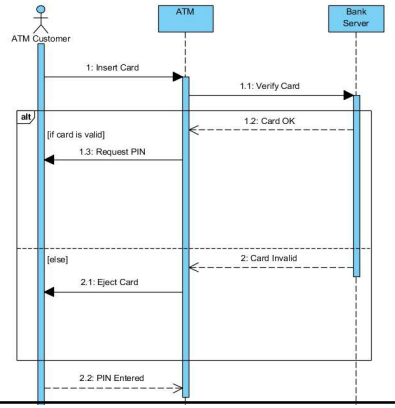
- In a functioning system
  - Objects interact with one another during its' execution
- Sequence Diagrams shows the time-based dynamics of the interaction
  - traces the execution of a scenario in the same context
    - To a large degree, a sequence diagram is simply another way to represent an object diagram (we'll discuss object diagrams latter)

29




30

## Sequence Diagram (Cont..)



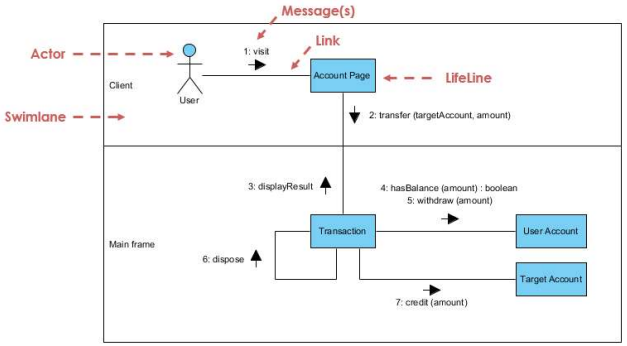

31

## Communication Diagram

- A communication diagram is a type of interaction diagram that focuses on
  - How objects are linked and the messages they pass, as they participate in a specific interaction
    - More focused on showing the collaboration of objects

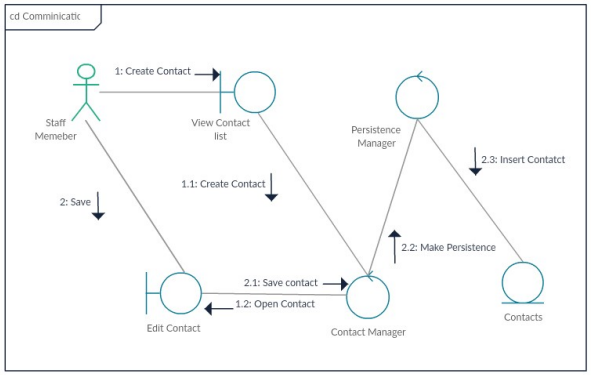
32

## Communication Diagram (Cont..)




33

## Communication Diagram (Cont..)



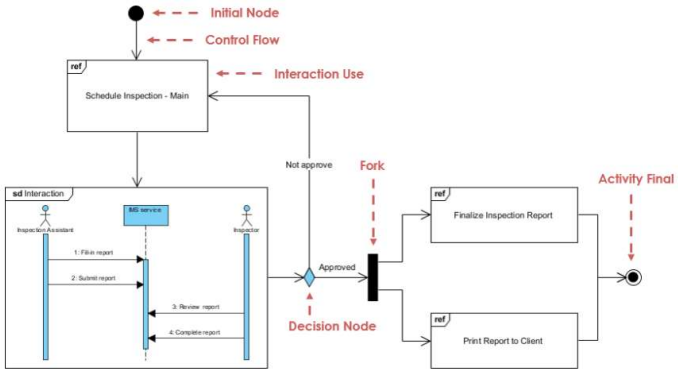

34

## Interaction Overview Diagram

- A combination of;
  - Activity diagrams
  - Sequence diagrams
- Intended to provide an overview of the flow of control between diagram elements

35

## Interaction Overview Diagram (Cont..)



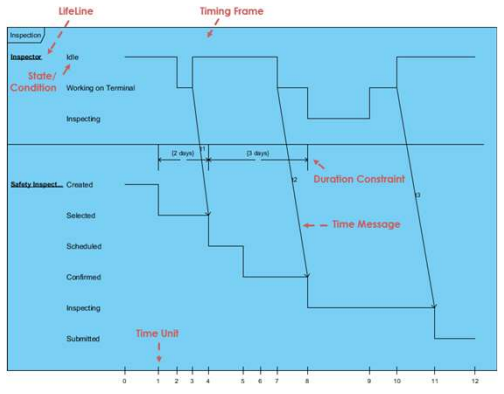

36

## Timing Diagram

- The purpose of a Timing Diagram is to show
  - How the states of an element or elements change over time
  - How events change those states

37

## Timing diagram (Cont..)




38

## STRUCTURAL DIAGRAMS

39

## Structural Diagrams

- Class diagram
- Component diagram
- Package diagram
- Object diagram
- Deployment diagram

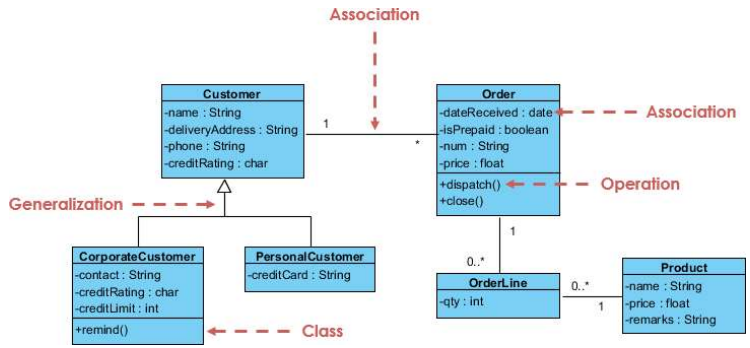
40

## Class Diagram

- A class diagram shows the existence of classes and their relationships in the logical design of a system.
- During analysis, class diagrams indicate the common roles and responsibilities of the entities that provide the system's behavior.
- During design, class diagrams capture the structure of the classes that form the system's architecture.

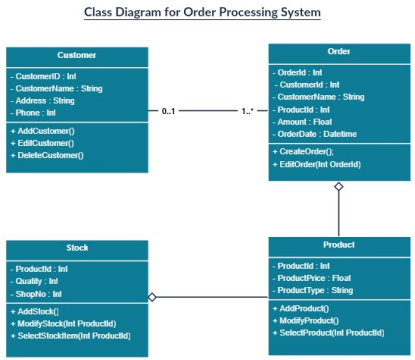
41

## Class Diagram (Cont..)




42

## Class Diagram (Cont..)



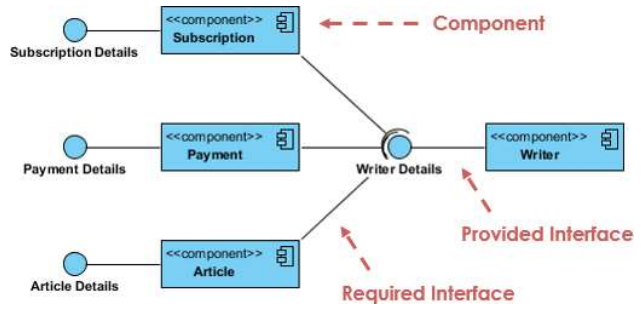

43

## Component Diagram

- A component represents
  - a reusable piece of software that provides some meaningful aggregate of functionality
- A component diagram shows
  - the internal structure of components and their dependencies with other components
- This diagram provides
  - the representation of components, collaborating through well-defined interfaces to provide system functionality

44

## Component Diagram (Cont..)



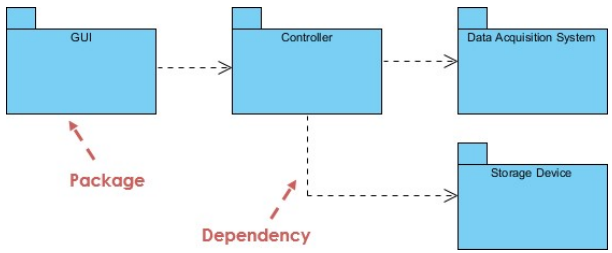

45

## Package Diagram

- Packages
  - Sometimes, its needed to organize the elements of a diagram into a group
- Need to show that a number of classes or components are part of a particular subsystem
- To do this,
  - Group them into a package represented by a tabbed folder

46

## Package Diagram (Cont..)



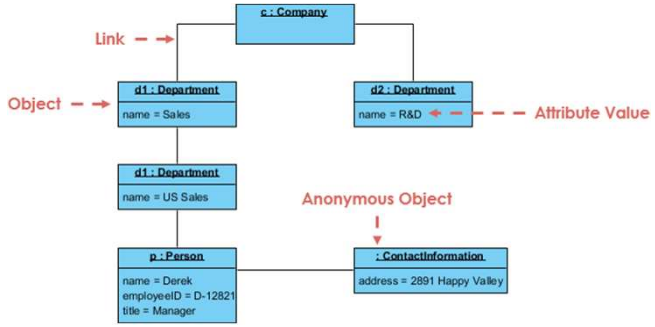

47

## Object Diagram

- An object diagram shows the existence of objects and their relationships in the logical design of a system.
- A single object diagram represents a view of the object structure of a system and is typically used to represent a scenario.
- It shows a complete or partial view of the structure of a modeled system at a specific time.

48

## Object Diagram (Cont..)



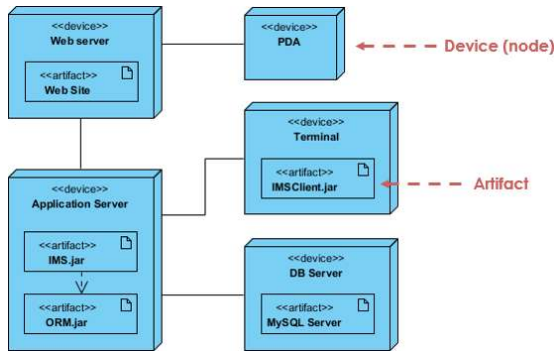

49

## Deployment Diagram

- A deployment diagram shows the allocation of artifacts to nodes in the physical design of a system.
- A single deployment diagram represents a view into the artifact structure of a system.
- During development, we use deployment diagrams to indicate the physical collection of nodes that serve as the platform for execution of our system.

50

## Deployment Diagram (Cont..)




51

## Common Uses of UML Diagrams

**Class Diagrams**

- Model the vocabulary of the system
- Model simple collaboration
- Model a logical database schema

**Object Diagrams**

- Model Object Structures
- Primary support of the functional requirement of the system

**Use Case Diagrams**

- Model the context of the system
- Model requirements of the system

52

## Common Uses of UML Diagrams (Cont..)

**Sequence Diagrams**

- Model interaction among object in time sequence

**Collaboration Diagrams**

- Model flow of control by the organization

**Activity Diagrams**

- Model a workflow of a single use case or for the entire software

53

## Why So Many Diagrams?

- The UML diagrams make it possible to examine a system from a number of viewpoints.
- It's important to note that not all the diagrams must appear in every UML model.
- It's accepted to use a subset of above UML diagrams in describing a system.

54

## Why Many Views Of A System?

- A system has a number of different stakeholders (people who have interest in different aspects of the system).
- Eg:
  - If you're designing a washing machine's motor, you have one view of the system
  - If you're writing the operating instructions, you have another
  - If you're designing the machine's overall shape, you see the system in a totally different way than if you just want to wash your clothes
- Each UML diagram gives you a way of incorporating a particular view.
- The objective is to satisfy every type of stakeholder.

55

## END

56